

In this notebook, we will explain in details the theory of xgboost tree models. The xgboost model is an ensemble of decision trees. The training process of xgboost model is to minimize a certain loss function.

1 How do we train a multi-class classification model?

In this section, we will focus on the multi-class classification problem trained with loss function being logloss.

1.1 Notation

N	number of samples
d	number of features
K	number of classes in the classification problem
x_i	feature vector of sample i , $x_i \in \mathbb{R}^d$
y_i	one-hot label vector of sample i , $y_i \in \{0, 1\}^K$
$\mathcal{F}$	the space of all possible decision trees.
f_t	the K trees added in iteration t , $f_t = (f_t^1, f_t^2, \dots, f_t^K) \in \mathcal{F}^K$, and $f_t : \mathbb{R}^d \rightarrow \mathbb{R}^K$
M_t	the tree model after iteration t , it is an ensemble of trees $f_1, \dots, f_t$.
θ	the set of all parameters in the model
$\hat{y}_i^t$	model prediction of x_i at t -th iteration, $\hat{y}_i^t \in \mathbb{R}^K$; if $t = 0$, it is the base value.
$\text{Grad}(L)$	gradient of function L , here with $L : \mathbb{R}^K \rightarrow \mathbb{R}$. $\text{Grad}(L) _{\hat{y}} \in \mathbb{R}^K$
$\text{Hess}(L)$	hessian of function L , here with $L : \mathbb{R}^K \rightarrow \mathbb{R}$. $\text{Hess}(L) _{\hat{y}} \in \mathbb{R}^{K \times K}$
$\cdot$	dot product of two vectors;
$\ \cdot\ _1$	L_1 norm of a vector; $\ (x_0, \dots, x_n)\ _1 = x_0 + \dots + x_n $
$\ \cdot\ _2$	L_2 norm of a vector; $\ (x_0, \dots, x_n)\ _2 = \sqrt{ x_0 ^2 + \dots + x_n ^2}$

1.2 Model and objective function

The objective function consists of two parts: training loss function (L) and regularization term (ω). At t th iteration, where the learning rate η is given, the model value given the feature input being x_i is

$$(\hat{y}_i^t)_{\text{modified}} = \hat{y}_i^{t-1} + \eta f_t(x_i)$$

instead of

$$\hat{y}_i^t = \hat{y}_i^{t-1} + f_t(x_i)$$

But at t th iteration, the objective function (no matter whether the learning rate $\eta = 1$ or not) is

$$\begin{aligned} obj(\theta) &= L(y, \hat{y}) + \omega(M_t) \\ &= \sum_{i=1}^N L(y_i, \hat{y}_i^t) + \omega(M_{t-1} + f_t) \\ &= \sum_{i=1}^N L(y_i, \hat{y}_i^{t-1} + f_t(x_i)) + \omega(f_t) + \omega(M_{t-1}) \end{aligned}$$

The newly added tree f_t is chosen to minimize the above objective function at t th iteration. We can use the second-order Taylor expansion to approximate the loss function:

$$\begin{aligned} obj(\theta) &\approx \sum_{i=1}^N \left[L(y_i, \hat{y}_i^{t-1}) + \text{Grad}(L)|_{\hat{y}_i^{t-1}} \cdot f_t(x_i) + \frac{1}{2} f_t(x_i)^T \text{Hess}(L)|_{\hat{y}_i^{t-1}} f_t(x_i) \right] + \omega(f_t) + \omega(M_{t-1}) \\ &= \sum_{i=1}^N \left[\text{Grad}(L)|_{\hat{y}_i^{t-1}} \cdot f_t(x_i) + \frac{1}{2} f_t(x_i)^T \text{Hess}(L)|_{\hat{y}_i^{t-1}} f_t(x_i) \right] + \omega(f_t) + \text{const} \end{aligned}$$

Notation: $\cdot$ here means the dot product of two vectors.

In the XGBoost package, the complexity of the tree $\omega(f)$ (or **regularization term**) is defined as

$$\begin{aligned} \omega(f) &= \gamma V + \frac{1}{2} \lambda \sum_{j=1}^V \|v_j\|_2^2 + \alpha \sum_{j=1}^V \|v_j\|_1 \\ &= \gamma V + \frac{1}{2} \lambda \sum_{j=1}^V v_j^T v_j + \alpha \sum_{j=1}^V \|v_j\|_1 \end{aligned}$$

where V is the number of leaves in the tree, $v_j \in \mathbb{R}^{K*1}$ is the score on leaf j . Parameters γ , λ and α are to control the regularization.

Now the tree is $f(x) = v_{q(x)}$ with $q: \mathbb{R}^d \rightarrow \{1, 2, \dots, V\}$ being the structure of the tree that maps a sample to the index of a leaf.

Therefore, the objective function for t th generation can be rewritten as

$$\begin{aligned} obj^t &= \sum_{i=1}^N \left[\text{Grad}(L)|_{\hat{y}_i^{t-1}} \cdot v_{q(x_i)} + \frac{1}{2} v_{q(x_i)}^T \text{Hess}(L)|_{\hat{y}_i^{t-1}} v_{q(x_i)} \right] + \gamma V + \frac{1}{2} \lambda^2 \sum_{j=1}^V v_j^T v_j + \alpha \sum_{j=1}^V \|v_j\|_1 \\ &= \sum_{j=1}^V \left[\left(\sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}} \right) \cdot v_j + \frac{1}{2} v_j^T \left(\lambda I + \sum_{i \in I_j} \text{Hess}(L)|_{\hat{y}_i^{t-1}} \right) v_j + \alpha \eta \|v_j\|_1 + \gamma \right] \end{aligned}$$

where $I_j = \{i | q(x_i) = j\}$ is the set of sample index assigned to leaf j .

Here, we have one assumption the objective function will be “convex”. In this case, the assumption is $\lambda I + \sum_{i \in I_j} \text{Hess}(L)|_{\hat{y}_i^{t-1}}$ being “**semi-definite**”, or further more $\text{Hess}(L)$ being semi-definite.

1.3 Tree growth process and leaf score calculation

The tree is built in a greedy manner. Starting from the root, we iteratively add branches to the tree. At each step, we try to split a leaf into two leaves. The split is determined by a feature and a threshold. For example, if we split leaf j using feature m and threshold c , then all samples in I_j with feature m value **less than** c will be assigned to the left child leaf, and the rest will be assigned to the right child leaf.

Assume j is a newly added leaf node from a split, the optimal leaf score v_j is the value minimizing above objective function for leaf j (“*derivative*” being 0).

1.3.1 Case: $\alpha = 0$

For simplicity, we assume the L_1 regularization term is 0. The derivative for the objective function for leaf j is

$$0 = \sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}} + (\lambda I + \sum_{i \in I_j} \text{Hess}(L)|_{\hat{y}_i^{t-1}})v_j$$

Thus the optimal leaf score is

$$\hat{v}_j = -(\lambda I + \sum_{i \in I_j} \text{Hess}(L)|_{\hat{y}_i^{t-1}})^{-1}(\sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}})$$

The objective function for leaf j after substituting the optimal leaf score is

$$\begin{aligned} obj_j^t &= -\frac{1}{2}(\sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}})^T \hat{v}_j + \gamma \\ &= -\frac{1}{2}(\sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}})^T (\lambda I + \sum_{i \in I_j} \text{Hess}(L)|_{\hat{y}_i^{t-1}})^{-1} (\sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}}) + \gamma \end{aligned} \quad (1)$$

The split is chosen to maximize the **gain**, which is defined as the reduction in the objective function after the split.

Suppose leaf j is split into two leaves j_L and j_R , then the loss difference is

$$\begin{aligned} \text{loss diff} &= obj_{before \text{ split}} - obj_{after \text{ split}} \\ &= obj_j^t - (obj_{j_L}^t + obj_{j_R}^t) \end{aligned}$$

After substituting the optimal score, the loss difference can be rewritten as

$$\begin{aligned} \text{loss diff} &= \frac{1}{2}(\sum_{i \in I_{j_L}} \text{Grad}(L)|_{\hat{y}_i^{t-1}})^T (\lambda I + \sum_{i \in I_{j_L}} \text{Hess}(L)|_{\hat{y}_i^{t-1}})^{-1} (\sum_{i \in I_{j_L}} \text{Grad}(L)|_{\hat{y}_i^{t-1}}) \\ &\quad + \frac{1}{2}(\sum_{i \in I_{j_R}} \text{Grad}(L)|_{\hat{y}_i^{t-1}})^T (\lambda I + \sum_{i \in I_{j_R}} \text{Hess}(L)|_{\hat{y}_i^{t-1}})^{-1} (\sum_{i \in I_{j_R}} \text{Grad}(L)|_{\hat{y}_i^{t-1}}) \\ &\quad - \frac{1}{2}(\sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}})^T (\lambda I + \sum_{i \in I_j} \text{Hess}(L)|_{\hat{y}_i^{t-1}})^{-1} (\sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}}) - \gamma \\ &=: \text{gain} - \gamma \end{aligned} \quad (2)$$

The tree growth process stops when the loss difference is less than 0, i.e gain $< \gamma$. This explains the name in XGBoost package, the parameter **min_split_loss** is γ .

1.3.2 Case: $\alpha > 0$ (L_1 regularization term is not 0)

When $\alpha > 0$, the optimal leaf score v_j is the solution to the following equation:

$$\vec{0} \in \sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}} + (\lambda I + \sum_{i \in I_j} \text{Hess}(L)|_{\hat{y}_i^{t-1}})v_j + \alpha \partial \|v_j\|_1$$

where $\partial \|v_j\|_1$ is the subdifferential of $\|v_j\|_1$.

To save notation, we set $w := \sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}}$, $A := (\lambda I + \sum_{i \in I_j} \text{Hess}(L)|_{\hat{y}_i^{t-1}})$ and $v := v_j$. The problem is to find v such that the condition below holds

$$0 \in w + Av + \alpha \partial \|v\|_1$$

Notice that in XGBoost package, **only the diagonal elements** of the Hessian matrix are considered, plus the “semi-definite” condition above, we have $A_{kk} > 0$ (If $\text{Hess}(L)$ is semi-definite and $\lambda > 0$). Under this case (A being a diagonal matrix), there is a closed formula for the optimized leaf score.

The above condition can be considered coordinate-wise. We only consider k th coordinate and have several subcases:

1. case $v_k > 0$:

$$0 = w_k + A_{kk}v_k + \alpha \iff v_k = -A_{kk}^{-1}(w_k + \alpha) > 0$$

$w_k < -\alpha$ must hold.

2. case $v_k < 0$:

$$0 = w_k + A_{kk}v_k - \alpha \iff v_k = -A_{kk}^{-1}(w_k - \alpha) < 0$$

$w_k > \alpha$ must hold.

3. case $v_k = 0$:

$$w_k \in [-\alpha, \alpha]$$

Thus the optimal score $\hat{v}_k$ is

$$\hat{v}_k = \begin{cases} -A_{kk}^{-1}(w_k + \alpha) & w_k < -\alpha \\ 0 & w_k \in [-\alpha, \alpha] \\ -A_{kk}^{-1}(w_k - \alpha) & w_k > \alpha \end{cases}$$

From above closed formula, we can see that L_1 regularization term indeed tends to cause the optimal value being 0.

Define the **soft-thresholding** operator, denoted by $\mathcal{S}(\cdot; \alpha)$ to be

$$\mathcal{S}(w; \alpha) = \text{sgn}(w)(|w| - \alpha)_+ = \begin{cases} w - \alpha & \text{if } w > \alpha \\ 0 & \text{if } |w| \leq \alpha \\ w + \alpha & \text{if } w < -\alpha \end{cases}$$

Thus the optimal score is

$$\hat{v}_k = -A_{kk}^{-1} \mathcal{S}(w_k; \alpha)$$

Similarly, we can compute the objective function (only consider the part contributing by k th coordinate) for leaf j given the optimal score:

$$\begin{aligned} obj_j^t(k) &= w_k \hat{v}_k + \frac{1}{2} A_{kk} \hat{v}_k^2 + \alpha |\hat{v}_k| \\ &= \frac{1}{2} \mathcal{S}(w_k; \alpha) \hat{v}_k \\ &= -\frac{\mathcal{S}(w_k; \alpha)^2}{2A_{kk}} \\ &= -\frac{\mathcal{S}((\sum_{i \in I_j} \text{Grad}(L)|_{\hat{y}_i^{t-1}})_k; \alpha)^2}{2(\lambda I + \sum_{i \in I_j} \text{Hess}(L)|_{\hat{y}_i^{t-1}})_{kk}} \end{aligned} \quad (3)$$

It aligns with the case when $\alpha = 0$, and the objective function is independent of η . The gain from coordinate k is defined as

$$\text{gain}(k) = \text{obj}^t(k)_{\text{parent}} - (\text{obj}^t(k)_{\text{left child}} + \text{obj}^t(k)_{\text{right child}}) \quad (4)$$

1.4 Special case: multi-class classification with softmax logloss

We calculated in details the gain in this special case. The raw output for classification actually is not the probability. The output is the raw score for each class, which is then converted to probability using softmax function. Here we assume $\hat{y}$ is the raw score, then the loss function is the softmax logloss: The probability is calculated as

$$\hat{p} = \frac{e^{\hat{y}}}{\sum_{j=1}^K e^{\hat{y}_j}} = \frac{1}{\sum_{j=1}^K e^{\hat{y}_j}} (e^{\hat{y}_1}, \dots, e^{\hat{y}_K}) = (\hat{p}_1, \dots, \hat{p}_K) \in \mathbb{R}^K$$

The loss function is defined as

$$\begin{aligned} L(y, \hat{y}) &= \sum_{k=1}^K y_k \log(\hat{p}_k) = -\sum_{k=1}^K y_k \log \left(\frac{e^{\hat{y}_k}}{\sum_{j=1}^K e^{\hat{y}_j}} \right) \\ &= -\sum_{k=1}^K y_k \left(\hat{y}_k - \log \left(\sum_{j=1}^K e^{\hat{y}_j} \right) \right) \\ &= -\sum_{k=1}^K y_k \hat{y}_k + \log \left(\sum_{j=1}^K e^{\hat{y}_j} \right) \end{aligned}$$

$$\text{Grad}(L)|_{\hat{y}} = \left(\frac{\partial L}{\partial \hat{y}_1}, \dots, \frac{\partial L}{\partial \hat{y}_K} \right) = -y + \hat{p} = (\hat{p}_1 - y_1, \dots, \hat{p}_K - y_K) \in \mathbb{R}^K$$

$$\text{Hess}(L)|_{\hat{y}} = \left(\frac{\partial^2 L}{\partial \hat{y}_i \partial \hat{y}_j} \right)_{i,j=1}^K = \begin{cases} \hat{p}_i(1 - \hat{p}_i) & \text{if } i = j \\ -\hat{p}_i \hat{p}_j & \text{if } i \neq j \end{cases}$$

Attention 1. *The implementation of Hessian matrix for softmax logloss has an extra constant factor 2. Check [here](#) for details. Respectively the loss functions as in (??) and (??) do **not** have the factor $\frac{1}{2}$.*

Notice in XGBoost package, the Hessian matrix is assumed to be diagonal, thus the objective function (??) can be realized as the sum of the coordinate-wise objective function. Instead of growing one tree at each iteration for all coordinates, there is a new tree for each coordinate. For example, if the number of class in the classification problem is 4, there will be 4 new trees for each iteration. Also the parameter γ is applied to **each tree**. Because the gradient and hessian of i th coordinate also depends on other coordinate, thus we have the observation below:

Observation 1. *Fitting a multi-class classification problem with K classes using softmax logloss function is **different** from fitting K binary classification models separately.*

Detailed tree growing process:

1. Fix a coordinate and start with base value, use (??) to get the objective function for that coordinate, then range through a list of features and a list of splits to compute the gain (??). If $\text{gain} \leq 0$, we will stop the split; else we have the split.
2. To be more precise on above process when `tree_method` is `exact`, for each feature, sort the feature values, all the possible split values are the list of midpoints of consecutive feature values. Then range over all the possible split values for all features. Notice that there is extra subtlety if there is missing values in the feature, the split can also happen such that missing and no-missing are two children nodes.
3. repeat the process for above left and right nodes separately, also for each coordinate.
4. Stop the tree growing process until the iteration number has reached the max or some early stopping criterion is satisfied.

2 Terminology/hyperparameters used in Xgboost package

Here we mainly focus on the [classification model](#) in scikit-learn API. Here we only explain some

- `n_estimators`: Number of boosting rounds. For a multi-class classification problem, the final number of trees is $K * \text{num of iteration rounds}$.
- `max_depth`: Maximum depth of a tree.
- `max_leaves`: Maximum number of leaves in one tree allowed. It is used only when `grow_policy=lossguide`.

- **grow_policy**: Choices are **depthwise** and **lossguide**.
depthwise splits first at nodes closest to the root. Combined with **max_depth** to control the size of tree.
lossguide splits first at nodes with highest loss change.
- **booster**: which booster to choose. For tree based models, **gbtree** and **dart** are the options. **gblinear** uses linear function. For more details above **dart**, check the xgboost official [page](#).
- **tree_method**: **exact** is the most accurate but time-consuming way, it enumerates all the possible splits on all the features. **approx** creates a list of candidate splits based on the Hessian. **hist**: Fastest. For details, check the [reference paper](#).
- **gamma**: γ appeared in the complexity of trees. The larger γ is, the higher requirement for splitting. It can be used to control overfitting.
- **min_child_weight**: Minimum sum of instance weight (Hessian) needed in a child. If the sum of weight $<$ **min_child_weight**, the building process will give up further splitting. The larger the number is, the higher requirement for splitting.
- **max_delta_step**: The maximal value allowed for a leaf node output. This limit help make the update step more conservative.
- **subsample**: $\in (0, 1]$, the ratio of training samples to fit trees in **EACH** booting round. It can prevent overfitting.
- **scale_pos_weight**: It is used in imbalanced binary classification. The gradient and Hessian for the positive class is multiplied by **scale_pos_weight** to increase/decrease the impact of the positive class on the final loss function. A typical value to consider is $\text{sum(negative instances)} / \text{sum(positive instances)}$; Not used in multi-class classification. The impact is similar to **sample_weight** in the **fit** method.
- **eval_metric**: If a list of metrics is given, the **LAST one** will be used for early stopping
- **eval_set**: one parameter for model fitting. If a list of data sets is given, the **LAST one** will be used for early stopping.

2.1 feature importance

There are several feature importance for tree models. They are **gain**, **weight**, **cover**, **total_gain** and **total_cover**.

- **gain**
 For each split of a tree, the gain is defined as above (??).
 The gain obtained using `get_booster().get_score(importance='gain')` is the gain per split. To get this number, compute the sum of gain for all splits using this feature, then the number of splits, the average is the gain from the package.

- **cover**

Cover is the hessian. In the tree of xgboost model, each node has its cover, which is the sum of Hessians of all samples in the node. It is independent of the feature on which the split is depending.

- **weight**

The weight of a given feature means the number of times the feature is used for split.

3 Appendix

3.1 Computation of subgradient of $\|x\|_1$

From the definition of [subgradient](#), the subgradient of $\|x\|_1$ at point a is the set of vectors v satisfying

$$\|x\|_1 - \|a\|_1 \geq v \cdot (x - a)$$

for all x .

The above condition is

$$\begin{aligned} |x_1| - |a_1| + \dots + |x_n| - |a_n| &\geq v_1(x_1 - a_1) + \dots + v_n(x_n - a_n) \\ \iff |x_i| - |a_i| &\geq v_i(x_i - a_i) \text{ for all } i \end{aligned}$$

Thus

$$v_i = \begin{cases} 1 & a_i > 0 \\ -1 & a_i < 0 \\ \in [-1, 1] & a_i = 0 \end{cases}$$